

Informe de Proyecto: TaskFlow - Aplicación de Gestión de Tareas

Módulo: Programación Avanzada en JavaScript

Desarrollador: Kisi Angélica Toledo Muñoz

0. Enlaces y Referencias

[LINK PRUEBA](<https://muss3t.github.io/taskflow/>)

[Repositorio del Proyecto en
GitHub](<https://github.com/tu-usuario/taskflow>)

[API REST de prueba utilizada:
JSONPlaceholder](<https://jsonplaceholder.typicode.com/>)

1. Resumen Ejecutivo

El presente documento detalla la arquitectura y las tecnologías implementadas en el desarrollo de **TaskFlow**, una aplicación web interactiva y modular construida íntegramente con JavaScript moderno (ES6+). El objetivo del proyecto es resolver la problemática de gestión de tareas mediante un enfoque escalable que integra Programación Orientada a Objetos (POO), manejo dinámico del DOM, asincronía y persistencia de datos con APIs externas.

2. Arquitectura y Orientación a Objetos (POO)

El proyecto se estructuró de manera modular separando las responsabilidades en distintos archivos lógicos:

- **Tarea.js**: Define la clase **Tarea**, la cual encapsula las propiedades individuales de cada ítem (**id**, **descripcion**,

`estado`, `fechaCreacion`, `fechaLimite`) y sus métodos de comportamiento (`cambiarEstado`, `eliminar`). Se utilizó `crypto.randomUUID()` para generar identificadores únicos.

- **GestorTareas.js**: Define la clase `GestorTareas`, encargada de administrar la colección de tareas mediante un arreglo, aplicando métodos para agregar, eliminar y filtrar elementos del estado global.

3. Implementación de Características ES6+

El código fue escrito respetando los estándares de ECMAScript 6 en adelante, reemplazando prácticas obsoletas por sintaxis moderna:

- **Declaración de variables**: Uso estricto de `let` y `const` para un manejo seguro del alcance (scope) de las variables.
- **Template Literals**: Implementados extensivamente en el archivo `ui.js` para inyectar variables directamente en las cadenas HTML (`${tarea.descripcion}`), mejorando la legibilidad al renderizar el DOM.
- **Arrow Functions**: Utilizadas para simplificar la sintaxis de las funciones, especialmente en el manejo de eventos y funciones de orden superior como `.map()`, `.filter()` y `.forEach()`.
- **Destructuring y Spread Operator**: Se aplicó *destructuring* en el constructor de la clase `Tarea` para recibir un objeto de configuración limpio, y el *spread operator* (`...`) en el gestor para crear copias inmutables del arreglo de tareas al agregar nuevos elementos.

4. Manipulación del DOM y Eventos

Toda la capa de interacción gráfica fue aislada en la clase `UI` (archivo `ui.js`). Se cumplieron los requisitos de interactividad mediante:

- **Eventos de Formulario (`submit`):** Captura de datos previniendo el comportamiento por defecto (`e.preventDefault()`).
- **Delegación de Eventos (`click`):** Aplicada al contenedor principal de la lista para detectar clics dinámicos en los botones "Completar" y "Eliminar", optimizando el rendimiento.
- **Interactividad Avanzada (`mouseover` / `mouseout`):** Implementada para cambiar el color de fondo de las tareas al pasar el cursor sobre ellas.
- **Búsqueda en Tiempo Real (`keyup`):** Creación de un filtro dinámico que evalúa y oculta/muestra las tareas del DOM conforme el usuario escribe en el campo de búsqueda.

5. Asincronía

Para garantizar una experiencia de usuario fluida y demostrar el dominio del ciclo de eventos (*Event Loop*), se implementaron las siguientes lógicas asíncronas:

- **`setTimeout`:** Utilizado para simular un retardo en la carga al momento de agregar una tarea y para ocultar automáticamente la notificación de éxito tras 2 segundos.

- **setInterval**: Implementado para crear un contador regresivo en vivo que calcula y actualiza cada 1 segundo el tiempo restante de las tareas que poseen una fecha límite.

6. Consumo de API y Persistencia de Datos

El proyecto integra el módulo `api.js` para gestionar los datos de manera asíncrona y persistente:

- **Consumo con `fetch` y Promesas**: Se desarrolló una función asíncrona (`async/await`) que se conecta a la API `JSONPlaceholder` para obtener un listado inicial de tareas en caso de que el usuario acceda por primera vez.
- **Manejo de Errores**: Se implementó un bloque `try/catch` para capturar y gestionar posibles fallos en la conexión de red durante el consumo de la API.
- **Almacenamiento Local (`localStorage`)**: Se crearon funciones para convertir el estado de la aplicación a formato texto (`JSON.stringify`) y guardarlo en el navegador del usuario, recuperándolo automáticamente al iniciar la aplicación (`JSON.parse`).

7. Conclusión

La aplicación **TaskFlow** cumple satisfactoriamente con la totalidad de los requerimientos técnicos y funcionales estipulados en la rúbrica de evaluación. La decisión de utilizar un patrón modular facilita el mantenimiento y futura escalabilidad del sistema, evidenciando un dominio práctico de la programación avanzada en JavaScript.